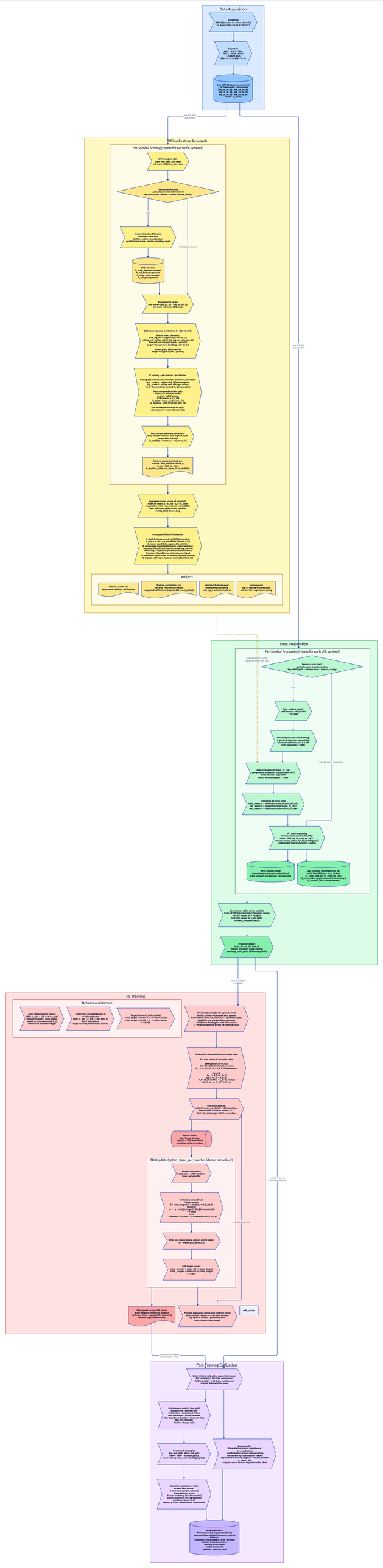

Data Acquisition
DataBento
MBP-10 (market-by-price, 10 levels)
us_hours filter (14:30–21:00 UTC)
6 symbols
AAPL · MSFT · TSLA
META · AMZN · AVGO
4 trading days
2026-02-25 to 2026-03-03
Raw MBP-10 parquet per symbol
~7M tick events · ~40 columns
bid_px_00..09 · ask_px_00..09
bid_sz_00..09 · ask_sz_00..09
bid_ct_00..09 · ask_ct_00..09
action · ts_event
raw tick data
per symbol
raw tick data
per symbol
Offline Feature Research
Per-Symbol Scoring (repeat for each of 6 symbols)
Chronological split
train: first train_size rows
val: next validation_size rows
Feature cache check
.cache/feature_transformation/
key = MD5(path + mtime + sizes + feature_config)
miss
hit (skip transform)
FeaturePipeline.fit(train)
transform train + val
Welford online normalisation
per-feature z-score · session boundary reset
Write to cache
fr_train_features.parquet
fr_val_features.parquet
fr_train_price.parquet
fr_val_price.parquet
Resolve price series
mid-price = (bid_px_00 + ask_px_00) / 2
(no close column in LOB data)
Build proxy targets per horizon h ∈ [10, 50, 200]
Sharpe proxy (default):
tick_log_ret = log(price[t] / price[t-1])
rolling_vol = rolling std of tick_log_ret (window=50)
forward_ret = log(price[t+h] / price[t])
target = forward_ret / (rolling_vol + 1e-10)
Return proxy (alternative):
target = log(price[t+h] / price[t])
IC scoring — per feature × per horizon
Rolling Spearman rank correlation (window_size=1000)
feat_ranked = rolling rank of feature values
tgt_ranked = rolling rank of target values
IC_t = Pearson(feat_ranked_t, tgt_ranked_t)
Stats computed on train split:
mean_ic = mean(IC series)
ic_std = std(IC series)
ICIR = mean_ic / ic_std
ic_tstat = mean_ic / (ic_std / √n)
ic_positive_ratio = fraction of IC > 0
Out-of-sample check on val split:
val_mean_ic = mean IC on val data
Best horizon selection per feature
keep stats for horizon with highest |ICIR|
record best_horizon
ic_stability = |mean_ic − val_mean_ic|
feature_scores_{symbol}.csv
feature · best_horizon · mean_ic
ic_std · ICIR · ic_tstat
ic_positive_ratio · val_mean_ic · ic_stability
Aggregate scores across all 6 symbols
mean of: mean_ic · ic_std · ICIR · ic_tstat
ic_positive_ratio · val_mean_ic · ic_stability
best_horizon = mode across symbols
sort by |ICIR| descending
Greedy conditional IC selection
1. Walk features sorted by |ICIR| descending
2. Skip if |ICIR| < icir_threshold (default 0.02)
3. Accept candidate → append to selected
4. Residualise remaining features against selected:
regressor.fit(selected_matrix, remaining_matrix)
remaining -= regressor.predict(selected_matrix)
(removes shared linear variance so next pick
is not a near-duplicate of an already-selected feature)
5. Repeat until top_k features selected (default 10)
Artifacts
feature_scores.csv
aggregated rankings · all features
feature_correlations.csv
pairwise Pearson correlation
on pooled val features (capped 20k rows/symbol)
selected_features.yaml
reduced feature config
only top_k selected features
summary.md
top-10 ranked features table
selected list · experiment config
Data Preparation
Per-Symbol Processing (repeat for each of 6 symbols)
reduced feature config
(optional, feeds next run)
Feature cache check
.cache/feature_transformation/
key = MD5(path + mtime + sizes + feature_config)
miss
hit (skip load + transform)
load_trading_data()
read parquet · drop NaNs
~7M rows
Chronological split (no shuffling)
train: first train_size rows (200k)
val: next validation_size (~3.4M)
test: remainder (~3.4M)
FeaturePipeline.fit(train_df_raw)
compute normalisation stats on train ONLY
Welford online algorithm
resets at session gaps > 1 hour
Transform all three splits
train_features = pipeline.transform(train_df_raw)
val_features = pipeline.transform(val_df_raw)
test_features = pipeline.transform(test_df_raw)
HFT post-processing
ensure_close_column_for_hft()
close = (bid_px_00 + ask_px_00) / 2
ensure_unique_index_for_hft_tradingenv()
deduplicate timestamps with 1µs gap
Write parquet cache
.cache/feature_transformation/{key}/
train.parquet · val.parquet · test.parquet
save_symbol_memmap(train_df)
write float32 binary array to disk
{i}_train_data.npy (n_rows × n_cols)
{i}_train_index.npy (nanosecond timestamps)
{i}_columns.json (column names)
Concatenate splits across symbols
train_df = first symbol only (streaming mode)
val_df = concat all 6 val splits
test_df = concat all 6 test splits
validate_prepared_data()
PreparedDataset
train_df · val_df · test_df
feature_columns · price_column
memmap_train_paths (6 MemmapPaths)
MemmapPaths
6 symbols
val_df + test_df
concatenated splits
RL Training
Network Architecture
Actor (deterministic policy)
MLP: [n_obs → 128 → 64 → n_act]
ReLU activations · Tanh output
scaled to action bounds [-1, 1]
continuous portfolio weight
Twin Critics (clipped double Q)
2 × ValueOperator
MLP: [n_obs + n_act → 128 → 64 → 1]
ReLU activations
input = concat(observation, action)
Target Networks (soft-copied)
actor_target ← τ·actor + (1−τ)·actor_target
critic_target ← τ·critic + (1−τ)·critic_target
τ = 0.001
StreamingTradingEnvXY (episode reset)
Shuffle symbol deck → pick next symbol
Pick random start ∈ [0, train_size − episode_length]
Load 50k-row window from memmap
Build inner TradingEnv with DSR reward
→ ~40 episodes total across 2M training steps
Differential Sharpe Ratio reward (per step)
R_t = log return of portfolio value
EMA updates (η = 0.01):
A_t = (1−η)·A_{t−1} + η·R_t (mean)
B_t = (1−η)·B_{t−1} + η·R_t² (2nd moment)
Reward:
ΔA_t = R_t − A_{t−1}
ΔB_t = R_t² − B_{t−1}
D_t = (B_{t−1}·ΔA_t − A_{t−1}·ΔB_t/2)
/ (B_{t−1} − A_{t−1}²)^(3/2) + ε
SyncDataCollector
collect frames_per_batch = 200 transitions
exploration: Gaussian noise σ = 0.1
first init_rand_steps = 1000 are random
Replay Buffer
LazyTensorStorage
capacity = 100k transitions
sampling: uniform random
TD3 Update (optim_steps_per_batch = 5 times per collect)
Sample mini-batch
batch_size = 128 transitions
from replay buffer
Critic loss (smooth L1)
Target action:
ã = actor_target(s') + clip(N(0, σ=0.2), ±0.3)
Target Q:
y = r + γ · min(Q1_target(s',ã), Q2_target(s',ã))
γ = 0.995
Loss:
L = SmoothL1(Q1(s,a) − y) + SmoothL1(Q2(s,a) − y)
continue training
Actor loss (every policy_delay = 2 critic steps)
L = −mean(Q1(s, actor(s)))
Soft target update
actor_target ← τ·actor + (1−τ)·actor_target
critic_target ← τ·critic + (1−τ)·critic_target
τ = 0.001
Checkpoint (every 100k steps)
actor weights · twin critic weights
optimizer state · replay buffer (optional)
saved to logs/{experiment}/
Periodic evaluation (every eval_interval steps)
deterministic rollout on train-split window
log episode reward · portfolio return
market return benchmark
soft_update
trained actor weights
(deterministic mode)
Post-Training Evaluation
Deterministic rollout (no exploration noise)
full val split (~3.4M ticks, continuous)
full test split (~3.4M ticks, continuous)
actor in deterministic mode
Performance metrics (per split)
Sharpe ratio · Sortino ratio
Total return · Annualised return
Max drawdown · Avg drawdown
Max drawdown duration · Recovery time
Win rate (hit rate)
Position change ratio
Explainability
Permutation feature importance:
for each feature:
shuffle feature values in observation
measure drop in cumulative reward
importance = reward_original − reward_shuffled
n_steps = 500
output: ranked feature importance bar chart
Benchmark strategies
Buy-and-hold · Short-and-hold
TWAP · VWAP · Random policy
(same time period, same starting capital)
Statistical significance tests
vs each benchmark:
t-test (two-sample, returns)
Mann-Whitney U test
Sharpe bootstrap (n=10k samples)
Sortino bootstrap (n=10k samples)
confidence level = 0.95
reports p-value · test statistic · conclusion
MLflow artifacts
Parameters: full ExperimentConfig
Metrics: all per-split performance metrics
Artifacts:
evaluation plots (reward curve · actions)
feature importance chart
statistical test report
model checkpoint
selected_features.yaml